Real-Time Detection and Repair of LLM Agent Failures

Sunny Dubey
sjkumardube@gmail.com

Abstract

LLM agents fail mid-episode—they loop, cascade tool errors, drift off their goal, fabricate results, or silently absorb corrupted content—and the standard remedy, judging every step with a second LLM, costs more than the agent itself. We ask how much failure detection is achievable from *observable step telemetry alone* (semantic embeddings of step output, token-level uncertainty, action metadata), using monitors that cost microseconds per step and train only on healthy runs. Starting from a one-class echo-state-network (ESN) ensemble with CUSUM alarms, we build and validate, on 2,823 committed agent episodes across three frameworks, three local agent models spanning two families (qwen2.5 7b/3b, llama3.1 8b), and a commercial API (gemini-2.5-flash), a sequence of increasingly capable monitors: (1) the ESN, which wins decisively when failures have temporal room to develop; (2) a calibrated ESN+Mahalanobis hybrid whose learned fusion weights track the deployment regime; and (3) a content-grounding telemetry channel that lifts the monitors' shared blind spot (content corruption) from 0.23 to 0.40 pooled content detection (up to 0.63 on the research collectors where the corruption is content-visible; honestly inert on frameworks whose corruption leaves result text unchanged), a $+0.175$ gain that holds at every seed, without degrading behavioral detection. Validation on organic (non-injected) failures reveals a taxonomy with an honest boundary: a completion check catches failures of omission (7/7 silent aborts), the monitors transfer only *weakly* to organic commission (1 of 3 fabrications) and rank the organic set at/below chance without recalibration, and plausible-value corruption requires external reference (the escalation layer's job). A pre-registered fabrication study is explicitly underpowered (2 hallucinations in the pre-registered 55 episodes, 9 across all 175 organic episodes and only 2 of those fabricated *inputs*, against a pre-registered minimum of 10) and makes no detection claim — the models rarely invent — so the fabrication class is covered by a deterministic numeric-grounding verifier rather than a statistical monitor. Both burdens — the per-deployment null and the residual false-alarm rate — motivate a complementary layer that carries neither: **deterministic verification**, which recomputes a run's stated total from the tool results that run actually received and confirms every required call was made, needing no null, no threshold and no calibration. Head-to-head on the same episodes and labels it catches 60% of failures (96% with the coverage check) at **0 of 63 false positives** against the monitor's 54% at 17%; it replicates on 120 later episodes at disjoint seeds scored frozen (54%, 93% with coverage, 0 of 64), transfers unchanged across model families to llama3.1:8b (**110 of 110 at 0 of 10**), and catches **26 of 26** provoked fabrications — the class on which a one-class monitor structurally cannot be scored. A third check validates tool results against the shapes their tool can return, flagging **0 of 1825** healthy episodes while catching 46% of injected context corruption, 215 of 218 within one step of onset. Detection is then closed into **repair**: each flagged run is rolled back to its last fact-gathering step and re-run live, recovering **45%** of failures against a 16% resampling control ($p = 0.0005$) and lifting net task success from **52% to 73%** for about one extra model call per run. The full system runs at $\sim$200 μs per step, three orders of magnitude below a judge call.

1 Introduction

An agent episode is a sequence of steps; each step emits observable telemetry: what the agent said, how confident its tokens were, what tools it called, what they returned, how long everything took.

The monitoring question is whether a lightweight, always-on watchdog over this stream can raise a calibrated alarm at derailment onset—steps before the task fails or the budget burns—without model internals, without labels, and without a second LLM in the loop [1].

This work answers that question constructively, and documents with equal care what such a watchdog cannot do. Contributions:

1. **Real-ecosystem validation at scale, and it is the paper's spine**: 2,823 episodes over 25 datasets, of which 770 use real tools with live agents across bespoke, LangGraph and AutoGen loops, spanning three local agent models in two families and the gemini-2.5-flash API. Per-class coverage across nine real corpora is reported in full, including where it is poor.
2. **A controlled testbed with ground-truth onsets** (failure injector, 43-dim telemetry) and an 11-monitor comparison under matched false-alarm budgets, used to expose detector *mechanics* — ground-truth onsets at arbitrary horizons, and classes the real corpora do not contain. Its telemetry is constructed, so no deployment claim rests on it.
3. **A diagnosis of when temporal monitoring pays**: the ESN's detection advantage over a 50×-cheaper Mahalanobis baseline is a monotone function of post-onset horizon.
4. **A calibrated hybrid** whose supervised fusion weights learn which regime a deployment is in.
5. **A content-grounding telemetry channel** that closes the content blind spot (malformed JSON $0.07 \rightarrow 0.93$) with behavioral detection improved.
6. **Fusion lessons**: per-stream "dual-budget" thresholds with a train-max trip restore detections destroyed by shared thresholds.
7. **Organic-failure validation with a pre-registered replication**: monitors calibrated on healthy runs and then applied unchanged catch only **1 of 3** organic fabrications and rank the organic failure set at/below chance (within-organic AUROC 0.31–0.42), while a temperature-matched null still false-alarms on 36% of healthy episodes — so a dedicated pre-registered fabrication study is explicitly **underpowered** (2 genuine hallucinations in 55 episodes vs a pre-registered minimum of 10) and makes no detection claim, and the fabrication class is instead delivered by a deterministic numeric-grounding verifier. Monitors are blind to silent aborts (a trivial completion check catches 7/7), and the healthy null is highly specific to the deployment configuration.

2 Related Work

Change detection and one-class anomaly detection. Our alarms are classical sequential change detection: CUSUM [2] over per-channel surprise, in the framework of abrupt-change detection [3]. The monitors are one-class [4]: fit on healthy episodes only, no failure labels. Baselines follow this tradition—Mahalanobis-distance detectors, standard in out-of-distribution detection [5, 6], and isolation forests [7]. The temporal backbone is an echo-state network [8]: reservoir computing [9] gives an untrained recurrent feature map whose readout fits in closed form, which is what makes per-deployment recalibration — which §5 shows is mandatory, not optional — cheap. Confidence calibration is evaluated with expected calibration error [10].

Agent failure: taxonomy and attribution. Agent loops interleave reasoning and tool calls [11], and are deployed through frameworks such as AutoGen [12]; benchmarks show failure is common and heterogeneous [13]. Recent work has made that heterogeneity concrete: MAST derives an empirical taxonomy of 14 failure modes from 1600+ annotated traces across seven frameworks, grouped into specification, inter-agent misalignment, and task-verification failures [14]. Our four injected classes are a deliberately narrow, mechanically-verifiable slice of that space — we make no claim to cover it. A second line asks *which* agent and step caused a failure after the fact, and finds it hard: automated attribution reaches 53.5% accuracy on the responsible agent but 14.2% on the responsible step [15]. That problem is post-hoc and outcome-conditioned; ours is the online,

pre-outcome question — raise an alarm mid-episode, before the failure is known to have occurred — so step-level attribution results bound what our lead-time claims should be read to mean.

Runtime monitoring and guardrails. The closest systems intervene during execution rather than scoring afterwards. AgentSpec enforces hand-written rules (trigger, predicate, enforcement) at the agent's action boundary [16]; ProbGuard abstracts executions into symbolic states, learns a discrete-time Markov chain over them, and escalates when the predicted probability of reaching an unsafe state crosses a threshold [17]. Both require a specification of what "unsafe" means — a rule set or a labelled unsafe-state set. Our monitor is label-free and specification-free: it fits only on healthy episodes and alarms on deviation from that null, which is why it transfers badly across deployments (§5) but needs no rule authoring for a new one. We view these as complementary layers, not competitors: a rule engine catches the violations you can name, a one-class monitor is what remains for the ones you cannot.

LLM-as-judge. The dominant quality-control pattern is LLM-as-judge [1], now extensively surveyed [18]: accurate but costing a model call per check. Judges are also not neutral instruments — position bias alone varies systematically with judge, task, and quality gap across 15 judges and 22 tasks [19]. Both facts push the same way for us: we ask what a label-free monitor three orders of magnitude cheaper can and cannot catch, and use the judge as the escalation layer rather than the default. Our judge operating point is stipulated (TPR 0.90, FPR 0.02) rather than measured, which the bias literature suggests is the assumption most in need of empirical replacement.

Hallucination detection. Hallucination in generation is well surveyed [20], and agent-specific taxonomies now separate fabrication in the reasoning, tool-call, and observation stages of the loop [21]. Detection methods typically need model access or repeated sampling—self-consistency across stochastic samples [22], semantic entropy [23], or probes that approximate semantic entropy from the hidden states of a single generation [24]. Tool-using agents admit a cheaper, deterministic route: every figure the agent asserts should trace to a tool result it received. Our numeric-grounding verifier (§9) is this provenance check; it needs no sampling, no model access, and no ground-truth answer.

Internal-state monitoring. A parallel line monitors activations rather than behaviour: linear probes on the residual stream detect strategic deception that the model's outputs conceal [25]. That work is white-box — it needs the weights — where our telemetry is collected at the adapter boundary and works against an API. The two also fail in similar ways, which is the honest reading: probe accuracy degrades under distribution shift much as our monitors fail to transfer across agent models without recalibration (§5).

3 Telemetry

Each step t emits x_t, built in four causal generations:

Table 1: Telemetry vector evolution across versions.

Version	Dims	Content
v1 (base)	43	e_t: 32-dim hash embedding; u_t: 4 token-surprisal aggregates; m_t: action one-hot, log latency, outlen, error
v2	43	Tool RESULTS appended into step text so corrupted results reach semantic channel
v3 (+x)	51	8 derived behavior dims: cosine drift, task-anchor sim, tool success, retry, latency, context ratio, depth, self-cons.
v4 (+g)	60	9 content-grounding dims: query-result dissim, result self-cons, JSON-prefix validity, char anomaly, lexical flag

Run each as `py -m <package>.<module>`: the study runners live under `derail.experiments`, the analyses under `experimental` and `verification`.

The embedding is a deterministic char-3-gram feature hash (no model, no network). v4 telemetry costs 608 μs/step at the adapter; the lexical flag alone costs 2 μs/result.

4 Monitors

All monitors are one-class (fit on healthy episodes only) and causal. Alarm thresholds come from healthy-validation episode maxima at a fixed FA budget.

- **ESN ensemble + CUSUM**: per-channel reservoirs [8] predict the next step; normalized surprise and ensemble disagreement feed a one-sided CUSUM [2].
- **Baselines**: Mahalanobis and delta-Mahalanobis [5], cosine/self-drift, rolling entropy, linear AR, and trained GRU/LSTM/TCN sequence models.
- **Hybrids**: robust-z calibration of ESN and Mahalanobis streams, then weighted, max, or logistic fusion.
- **Grounded monitors**: memoryless robust-z max over the g channel joins as a third stream.
- **Completion check**: five lines checking if the final step is empty or a bare tool call.

5 Real-Ecosystem Validation: the Empirical Spine

Everything this paper claims about deployment rests on this section. The committed corpus spans 2,823 episodes over 25 datasets — three frameworks, three local agent models across two families, and the gemini-2.5-flash API — every episode a live agent run against real tools, with telemetry measured at the adapter rather than constructed.

Figure 1 shows the primary monitor on real streams: healthy runs stay two orders of magnitude below the alarm threshold, and each injected class alarms one step after its verified onset. Figure 2 gives the coverage picture across all nine real corpora, and it is the honest one. Looping and tool cascade are caught almost everywhere (0.48–1.00); goal drift is caught wherever it is present (0.66–0.86 on four of five corpora); and the genuine weak spots are the content-corruption classes, `wrong_document` (0.00–0.20) and `malformed_json` (0.05–0.50) — which is what motivates the grounding channel of §8, not a post-hoc rationalisation of it.

Two structural findings shape everything that follows. First, monitors do not transfer across deployments without recalibration: calibrated on qwen2.5:7b and deployed on llama3.1:8b over a matched task, tool and injector plan, the primary monitor sits at chance (AUROC 0.527, healthy FA 0.75), while the *same* target recalibrated on itself reaches 0.885 — so this is a property of the calibration, not of a hard corpus. Second, the memoryless delta-Mahalanobis wins on short-episode datasets, which the horizon diagnosis explains: over 716 injected episodes the ESN–Mahalanobis detection gap is **-0.09** when the post-onset horizon is $\leq$3 steps, **+0.23** at 4–8 and **+0.40** at $\geq$9 (correlation +0.37). The reservoir needs post-onset steps to integrate evidence; where it has them it wins, and where it does not the memoryless distance is the better monitor.

Breadth: a second model family and a commercial API. Recalibrated on itself, a llama3.1:8b corpus collected on an identical task, tool and injector plan reaches AUROC **0.885**, so the monitors are not specific to one model family. A lengthened Gemini corpus (125 episodes, 65 positives) reaches **0.794**: the approach works on a commercial API too. That corpus is gemini-2.5-flash on the same long research task, tool roster and injector plan as the qwen long corpus, so the provider is the only thing that differs. Its lower detection at a matched false-alarm budget (0.38 against 0.57) is a *horizon* effect, not a telemetry one — its positives are far shorter after onset (median 4 steps against 8), and at matched horizon the two agree (0.53 against 0.56). Ablating the token-surprisal channel inside four corpora that carry it moves the ESN by AUROC $+0.000$, so the deployment requirement is structured tool results, not logprob access.

The judge is measured, not assumed. A real gemini-2.5-flash judge on a labelled subset of 172 distinct prompts scores p_detect **0.548** (95% CI 0.44–0.65) and p_false **0.057**, against the 0.90/0.02 the escalation analysis assumes. Substituting the measured rates across all five master seeds leaves the call saving intact but drops detection recovery from 82% to **44%**, supported at zero of five seeds. Figure 3 shows why the layer is still worth having: judge and monitor fail on *different* classes — the

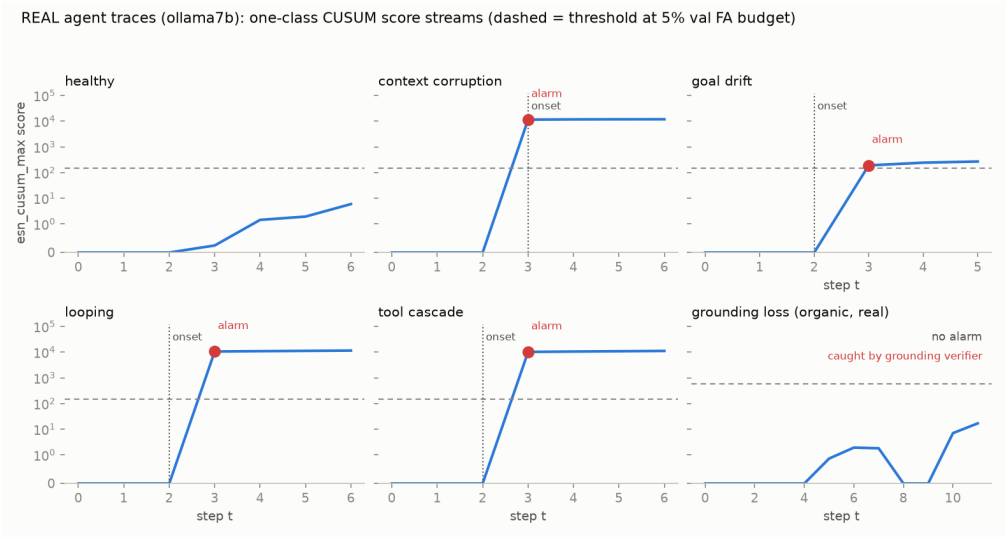

Figure 1: **Real agent traces** (`ollama7b`: qwen2.5:7b against real tools). One-class CUSUM score streams per failure class; dashed = threshold at the 5% validation FA budget, dotted = verified injection onset. The healthy run stays two orders of magnitude below the line, and every injected class — context corruption, *goal drift*, looping and tool cascade — alarms one step after onset. The sixth panel is *grounding loss*, and it is different in kind: fabrication cannot be injected into a live run, so this episode is a genuine one from the organic (non-injected) corpus, scored against that corpus's own healthy null. The CUSUM stays flat – a fabricated figure perturbs no behavioural channel – and the class is caught instead by the deterministic grounding verifier. That is the blind spot the verifier exists to cover, shown rather than asserted. Note the y axis is symlog: real streams span twelve orders of magnitude, because the CUSUM accumulates multiplicatively once a failure takes hold.

judge is perfect on goal drift and nearly blind on context corruption (0.18), where the monitor is the reverse (1.00).

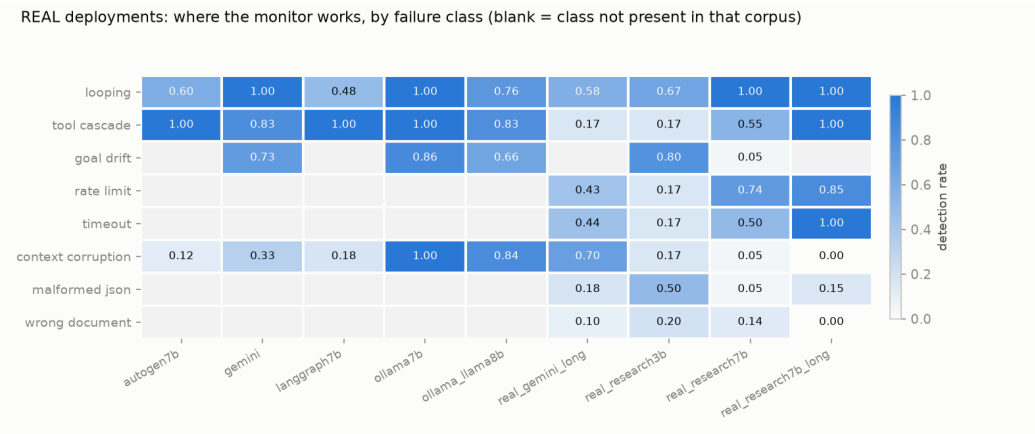

Figure 2: **Real deployments: where the monitor works, by failure class.** Primary-monitor detection rate, nine real corpora, blank where a corpus does not contain that class (absent and undetected are different claims). This is the honest coverage picture the simulator cannot give: looping and tool cascade are caught almost everywhere, *goal drift* is caught wherever it is present (0.66–0.86 on four of five corpora), and the genuine weak spots are the content-corruption classes — `wrong_document` 0.00–0.20 and `malformed_json` 0.05–0.50 — which is precisely what the grounding channel (§8) exists to address.

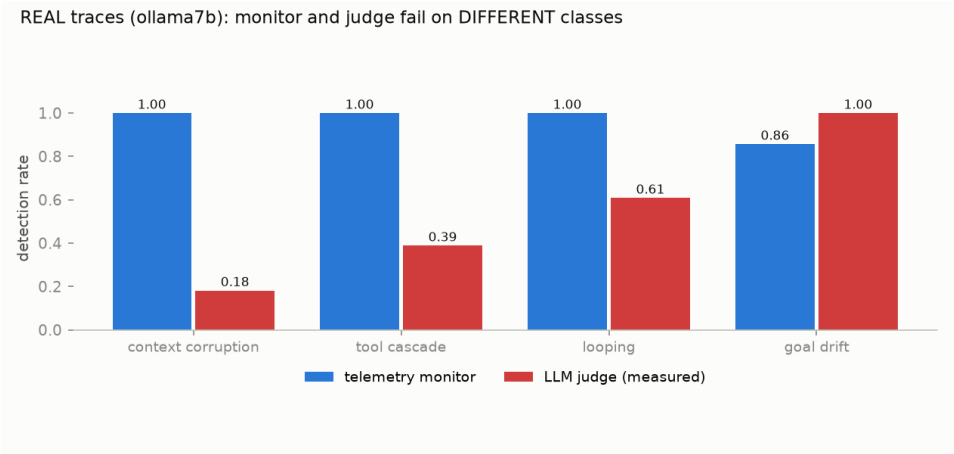

Figure 3: **Monitor and judge fail on different classes.** Both series are measured on the same real corpus: the judge rates from a live gemini-2.5-flash run on a labelled subset (172 distinct prompts), the monitor rates from the primary monitor on that corpus. The judge is perfect on goal drift and nearly blind on context corruption (0.18); the monitor is the reverse on context corruption (1.00). This is the empirical case for escalation as a *complementary layer* rather than a cheaper approximation of the judge.

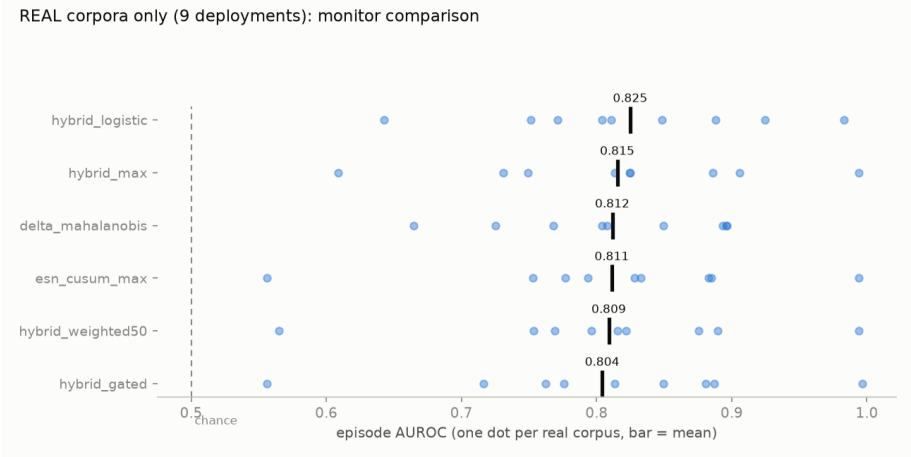

Figure 4: **Monitor comparison on the real corpora only.** Episode AUROC, one dot per real deployment and a bar at the mean; the simulator is excluded. The spread is the point: several per-dataset orderings that look decisive are ties once power is accounted for, so the defensible comparison is the pooled one rather than any single deployment.

6 Controlled Study: Detector Mechanics on Simulated Telemetry

This section is a mechanism study, and no deployment claim rests on it. The simulator *constructs the detector's input telemetry directly*: it writes the semantic-embedding state, token-uncertainty aggregates, action metadata, latency, output length, and error flags for each step, with no LLM, tokenizer, embedding measurement, or telemetry adapter in the loop. The class-channel signatures are therefore designed in, and what follows measures whether the monitors *recover* the injected structure — not whether that structure arises from a real model. It earns its place for two reasons the real corpora cannot supply: it provides ground-truth onsets at arbitrary horizons, and it contains classes the real corpora do not (grounding loss). Read it as a cartoon of the mechanism; the evidence is §5.

Its numbers should be read with that caveat attached, and one of them does not survive contact with measurement: the escalation result below assumes a judge we have since measured, and §5 reports what happens when the assumption is replaced. Over five master seeds the ESN detects **0.707** $\pm$ **0.068** of failures at a 4.4% realized FA with episode AUC **0.872** $\pm$ **0.015** and a mean lead of 4.6 steps (Figure 1); H1 holds at four of the five seeds and is honestly not supported at one. Label-free confidence is calibrated by the healthy-score null: its healthy stream is uniform to KS ≈ 0.12 (fused), and the oracle isotonic posterior (*with* labels) reaches ECE ≈ 0.03 [10]. A cost-optimal escalation policy (operating point selected on calibration) recovers **83% of judge-every-step detection at 8% of its judge calls** (master seed; H3b holds at four of five seeds).

H3b is conditional on the judge, and the assumption fails. That judge is a stipulated noisy oracle (p_detect 0.90, p_false 0.02). A real gemini-2.5-flash judge measured on a labelled subset of collected episodes (172 distinct prompts) scores p_detect **0.548** (95% CI 0.44–0.65) and p_false **0.057** (0.025–0.13) — both stipulated values lie outside their measured intervals. Substituting the measured rates and re-running all five master seeds with everything else fixed, the *call saving survives* (6% vs 8% of judge-every-step calls; cost ratio 0.63 vs 0.61) but detection recovery falls to **44%** (range 39–48%) and H3b is supported at **0 of 5 seeds**. The escalation architecture earns its cost saving; its detection claim should be read as 44%, not 83%. Per class the measured judge is perfect on goal drift (21/21) and nearly blind on context corruption (0.18) — the class the grounding channel targets — so judge and monitor fail on different classes.

7 The Calibrated Hybrid

Of 1,002 injected episodes, about a third of failures are visible to exactly one detector parent (ESN vs Mahalanobis: 242 ESN-only, 57 Mahalanobis-only, 403 both). The cross-fit logistic fusion recovers **100% / 66% / 79%** of the both/ESN-only/Maha-only cells and fires on just 1% of the cases neither parent flags. Its learned Mahalanobis weight share tracks the regime (from 0.38 on long horizons to 0.99 on short; Figure 5). The hybrid never statistically falls below the better parent at any tested seed.

8 Closing the Content Blind Spot

Behavioral and statistical monitors share one blind spot: corruption that changes *data* without changing *behavior*. The g channel adds nine causal content features, including a binary lexical relevance flag that completely separates off-topic retrievals.

9 Organic Failures and Conclusion

Thirty non-injected high-temperature (0.9) episodes yield 11 organic failures: 7 silent aborts, 3 fabricated counts, 1 ungrounded retrieval blend. Scored by monitors carrying the calibration they were deployed with, and not refit on this data, the behavioural monitors catch only **1 of the 3** fabrications and rank the whole organic failure set at/below chance (within-organic AUROC **0.31–0.42**); a temperature-matched null still false-alarms on **36%** of healthy episodes, because the organic

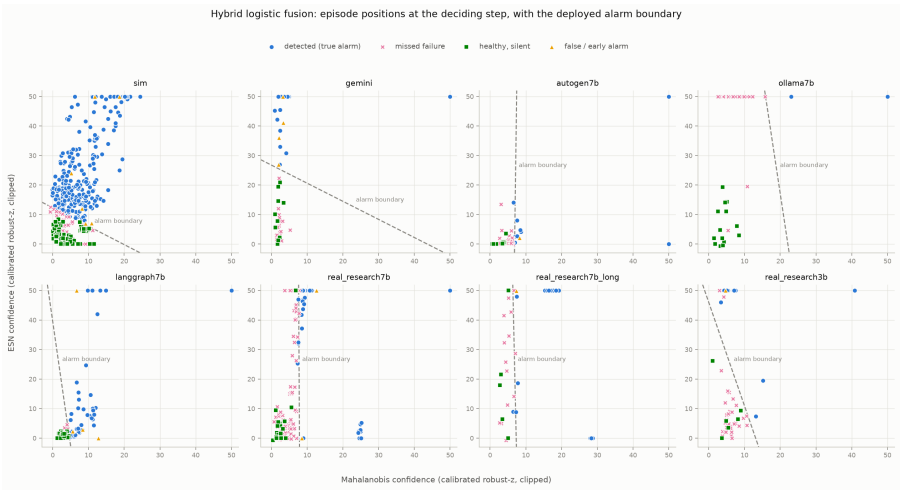

Figure 5: Hybrid logistic fusion: episode positions at the deciding step in the (Mahalanobis, ESN) calibrated-confidence plane, one panel per deployment, with the learned alarm boundary. The boundary's orientation is the learned regime: near-vertical (Mahalanobis-driven) on short-horizon real-agent datasets, ESN-driven on the long-horizon simulator.

Table 1: Detection rate (pooled injected episodes, n=602). Ungrounded → grounded; McNemar discordant (gained/lost) vs the ungrounded parent.

Fusion	Content (n=211)	Behavioral (n=391)
Content gate	0.23 → **0.40** (+37/-0)	0.73 → **0.80** (+25/-0)
Joint-budget	0.23 → **0.32** (+21/-1)	0.73 → 0.73 (+0/-3)
Logistic+g (labels)	0.14 → 0.17 (+14/-9)	0.74 → 0.69 (+5/-23)

runs were sampled at temperature 0.9. Detections at that false-alarm rate are not deployable evidence, and we report them as such. **Omission is invisible to telemetry but trivially fixable** (the five-line completion check catches 7/7 silent aborts at zero false positives), and **plausible-value corruption remains undetectable** without an external reference.

A **pre-registered replication** (design and analysis committed before data; $n = 55$ episodes at temperature 0.9) built the null cross-fit from the healthy subset of the same-temperature runs, but is **explicitly underpowered**: only 2 candidate hallucinations arise (pre-registered minimum 10), at a 36% healthy false-alarm rate, so it makes **no detection claim**. These well-aligned agent models rarely invent figures. Fabrication detection is therefore delivered not by a statistical monitor but by the mechanism that matches the failure: a deterministic per-step **numeric-grounding verifier** — every monetary figure the agent asserts must trace to a tool result it actually received, or an arithmetic combination thereof — which needs no ground-truth answer, runs online, flagged 0 false positives on the 25 healthy episodes, and catches an inserted ungrounded figure at the step it appears. The statistical monitors were never the right tool for this class; measuring that, and shipping the right tool, is the result.

Observable step telemetry covers most of the agent-failure space using microsecond-scale monitors — with failure classes it cannot cover delegated to mechanisms that can (completion check, grounding verifier, escalation to a judge) — providing a vital first line of defense before escalating to full LLM judges.

10 Deterministic Verification and Repair

The monitors above need a healthy null per deployment and still spend part of a false-alarm budget. A check needs neither. `total_consistency` recomputes a run's stated total from the tool results *that run actually received*; `required_coverage` confirms every call the task requires was made; `tool_contract` asks whether a result matched any shape its tool can return, and so reports at the step the result arrives rather than at the answer. None reads the hidden world the task was generated from, so all three are deployable as-is.

Table 2: Checks versus the behavioural monitor, same episodes and same objective labels (`verification_vs_monitor.csv`). Recall is comparable at the served temperature; the difference is precision.

	checks	monitor
failures caught, T=0.2 (served)	60% (**96%** with coverage)	54%
false positives, T=0.2	**0/63 = 0%**	11/63 = 17%
failures caught, T=0.9	65% (**96%** with coverage)	40%
false positives, T=0.9	**0/38 = 0%**	6/38 = 16%

The checks were written by inspecting failures in the serving arm, so that arm cannot also be their test set. A further 120 episodes at disjoint task seeds (40000+, zero overlap) were collected afterwards and scored with the checks frozen: 54% caught by totals, **93%** with coverage, arithmetic errors 36/36, and **0 of 64** false positives — the overall figure drops by the margin a genuine held-out test should cost, driven by the small `hallucinated` class (4/8). A llama3.1:8b arm on the *same* 120 task seeds, with nothing retuned, catches **110 of 110** failures at **0 of 10** false positives. On the provoked corpus the checks catch **26 of 26** fabrications; that corpus cannot score a one-class monitor at all, because provoking enough fabrication leaves 2 healthy episodes against the 15 a null needs. Scored across every labelled corpus, `tool_contract` trips on **0 of 1825** healthy episodes, flags 46% of `context_corruption` and 44% of `looping` and 0% of every other injected class, and where it fires is immediate: **215 of 218** flagged episodes within one step of onset.

Repair. Every flagged episode is rolled back to the same checkpoint and re-run under each repair rung, paired on the identical prefix and task; the rollback is real (a committed trace plus its seed rebuilds the conversation at step k, and every step after is a fresh model call), and success is graded by the study oracle, which the repair prompt never sees. Each cell is the mean of three independent repeats over $n = 55$ genuinely-wrong episodes.

Table 3: Repair rungs against a resampling control (`repair_policies.csv`). Asking for a re-check is what works, and naming the failing check works best.

rung	recovery	vs `resample`	calls/recovery
`none` — untouched	0%	—	—
`resample` — rollback + fresh sample	16%	*control*	14.7
`located` — + which check failed, no values	**45%**	$p = 0.0005$	6.4
`generic` — + a re-check instruction	36%	$p = 0.0347$	**5.8**
`specific` — + the finding, with values	36%	$p = 0.0192$	8.1
`recompute` — + use the calculator	28%	$p = 0.17$ (n.s.)	7.2
`adaptive`	21%	$p = 0.61$ (n.s.)	10.6

Net over all 120 episodes, charging each policy for any correct run it broke, `located` lifts task success from **52% to 73%**: 25 failures recovered and **zero** correct runs broken, because the checks flagged no already-correct episode. Retry luck is controlled for — plain resampling alone recovers 16%, so only the margin above that is credited to the repair. Two negative results sit inside this table and are kept rather than dropped. `recompute` routes the step to a calculator the agent is already

holding, which should fix the dominant arithmetic failure, and does not beat retry luck at this sample size ($p = 0.17$). And supplying the recomputed answer buys nothing: 26 of 55 `specific` hints contain the correct total outright, while `located` states no value at all and recovers at least as much — so the recovery is not coming from being handed the answer.

Repair coverage is real but partial, and the shape of the gap is measured. Over five injection classes $\times$ five task seeds with halting off (`alarm_repair.csv`, $n = 25$ live episodes) every one of the 18 behavioural alarms was followed by a repair attempt and no run that did not alarm was interrupted, but `goal_drift` is the only class a retry fixes (2 of 5). Where the tool layer itself is broken a retry fetches the same broken result: `tool_cascade` and `looping` escalate rather than recover, and the value of the intervention there is ending the episode fast — a loop trap exits at exactly 10 steps in 5 of 5 runs, against 30 before the circuit breaker. `grounding_loss` never alarms behaviourally (0 of 5) and is the grounding check's class by design. A contract violation is escalated, never repaired.

11 Limitations

We state the boundaries as measured, not as caveats. (1) **No cross-deployment transfer**: monitors do not transfer across frameworks, models, or even sampling temperature without recalibration; the healthy null must be collected under the exact serving distribution (temperature, toolset, probe pattern, latency regime) — three separate incidents in this study (latency drift, catalog contamination, temperature mismatch) were all this one failure. Measured across a model *family* on a matched task/tool plan (qwen2.5:7b $\rightarrow$ llama3.1:8b), transferred monitors sit at chance (AUROC 0.527, healthy FA 0.75) while the same target recalibrated on itself reaches 0.885 — so this is a property of the calibration, not of a hard target. The same experiment run *within* a family (qwen2.5:7b to qwen2.5:3b, `model_transfer.csv`) collapses further, to AUROC 0.29–0.38 — below chance — but is harder to read, because recalibrating on the 3b corpus itself only reaches 0.556, so "the target is simply hard to monitor" remains a live explanation there. The cross-family experiment is the cleaner one precisely because its target recalibrates to 0.885. (2) **Slow goal drift** evades every per-step-surprise monitor tested; only trajectory self-consistency sees it, weakly. This is a statement about *gradual* rotation, which is how the simulator constructs the class: each step differs little from the last, so no per-step surprise accumulates. Goal changes that are abrupt are caught — on real traces the same monitor detects goal drift at 0.66–0.86 on four of five corpora (Figure 2), and the measured judge catches it 21/21. The limit is the rate of change, not the class. (3) **Content corruption detection is bounded by telemetry completeness**: results not passed through the telemetry channel are invisible, and terse results carry too little mass to detect garbling reliably. The bound is on *structured tool results*, not on token logprobs: ablating the token-surprisal channel within four corpora that carry it (same episodes, splits, seeds, thresholds) moves the ESN by **AUROC** $+0.000$ **and detection** $+0.002$ on average, so a provider that rejects `response_logprobs` — as the gemini-2.5-flash tier used here does — loses almost nothing. The lower raw detection on that provider's corpus (0.38 vs 0.57) is a *horizon* effect, not a telemetry one: its positives are far shorter after onset (median 4 vs 8 steps, 28% at horizon ≤ 3 vs 2%), and at matched horizon ≥ 4 the two agree (0.53 at AUROC 0.902 vs 0.56). (4) **Plausible-value corruption** (a wrong-but-well-formed number) is undetectable from telemetry by construction and requires an external reference or judge. (5) **Fabrication claims are limited by base rate**: across both organic corpora the objective labeller flags 9 of 175 episodes as hallucinated (2 of 55 in the pre-registered corpus, 7 of 120 in the extension), and only **2 of those are fabricated *inputs*** — an ungrounded item figure, the class the grounding verifier targets — the other 7 being totals that match neither the truth nor any arithmetic derivation of grounded inputs. Both counts sit below the pre-registered minimum of 10, so no statistical detection claim for hallucination is made in either direction; the grounding verifier is validated on unit tests, healthy false-positive checks (0/25), and inserted-figure catches, not on organic fabrications. Under provocation the class becomes testable: making 20% of price-bearing tool calls fail *transiently* – the retry succeeds, so reporting a grounded total stays available and inventing the missing figure is the model's own choice – yields 11 ungrounded-input fabrications in 120 episodes, on which the verifier catches **0.55** with no false

positives on healthy episodes (0/9 here, 0/25 and 0/55 on the two unprovoked corpora). It is therefore specific but only about half sensitive, and that number exists only under provocation: unprovoked fabrication stays at 2 in 175 and supports no claim. (6) **Scope**: mock-tool and research-loop tasks, two local model families (qwen2.5, llama3.1) and one commercial API; wall-clock latency features are machine-specific and are excluded from the shipped local configuration.

Appendix

Artifact availability

All code, collected traces, result tables and figures are released together in the accompanying repository. Nothing in this paper is computed from data held back. The layout a reader needs is:

- `results/tables/` — every CSV and JSON named in Table 3. A claim in the text and the file named beside it are the same numbers; the file is the source.
- `results/figures/` — the figures in this paper, regenerated by `py -m derail.experiments.plots`.
- `traces/` — the agent episodes themselves, one JSONL file per episode with a `manifest.json` per corpus recording each episode's model, label, verified onset and checksum.
- `BASELINE_MANIFEST.json` — a SHA-256 for every source file, result artifact and trace in the repository. It is what makes the provenance in Table 3 checkable rather than asserted: `py -m devtools.artifact_manifest --check` recomputes every hash and reports any file that differs from the state these results were produced in.

Two further checks ship with the code. `py -m devtools.behavior_snapshot --check` re-runs the study at a fixed seed and compares every value against a stored baseline, so a change in behaviour cannot pass unnoticed; and `py -m pytest` runs the full suite, including the slow tests that exercise real tools. A container definition and a pinned lock file are included for a network-free CPU reproduction of the synthetic study.

Table 3: Provenance map: each claim to the artifact it is computed from. Every file listed is committed, and `BASELINE_MANIFEST.json` records a SHA-256 for it, so a reader can verify that the number in the text came from the file in the repository.

Claim	Artifact	Regenerate with
Real per-class coverage	`17b_per_class.csv`	`run_hybrid_study`
Cross-family transfer	`model_transfer_family.csv`	`run_model_transfer`
gemini-2.5-flash corpus	`gemini_long_*.csv`	`run_hybrid_study`
Measured judge	`judge_calibration_summary.json`	`run_judge_calibration`
Judge consequence for H3b	`judge_sensitivity.csv`	`judge_sensitivity`
Telemetry-channel ablation	`telemetry_dependence.csv`	`telemetry_dependence`
Recalibration cost	`recalibration_cost.csv`	`recalibration_cost`
Statistical power	`power_analysis*.csv`	`power_analysis`
Adversarial limit	`adversarial_evasion.csv`, `tamper_check.csv`	`tamper_check`
Organic + provoked fabrication	`organic_hallucination*.csv`, `provoked_fabrication.csv`	`score_provoked_fabrication`
Simulator study	`multiseed*.csv`, `h1_*`, `h3_*`	`run_multiseed`

References

[1] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.

[2] E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954.

[3] Michèle Basseville and Igor V. Nikiforov. *Detection of Abrupt Changes: Theory and Application*. Prentice Hall, 1993.

[4] Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):1–58, 2009.

[5] Kimin Lee, Kibok Lee, Honglak Lee, and Jinwoo Shin. A simple unified framework for detecting out-of-distribution samples and adversarial attacks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[6] Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *International Conference on Learning Representations (ICLR)*, 2017.

[7] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *IEEE International Conference on Data Mining (ICDM)*, pages 413–422, 2008.

[8] Herbert Jaeger. The "echo state" approach to analysing and training recurrent neural networks. Technical Report 148, GMD – German National Research Institute for Computer Science, 2001.

[9] Mantas Lukoševičius and Herbert Jaeger. Reservoir computing approaches to recurrent neural network training. *Computer Science Review*, 3(3):127–149, 2009.

[10] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, 2017.

[11] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[12] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[13] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024.

[14] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*, 2025.

[15] Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. *arXiv preprint arXiv:2505.00212*, 2025.

[16] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. In *IEEE/ACM International Conference on Software Engineering (ICSE)*, 2026. arXiv:2503.18666, 2025.

[17] Haoyu Wang, Christopher M. Poskitt, Jiali Wei, and Jun Sun. ProbGuard: Probabilistic runtime monitoring for LLM agent safety. *arXiv preprint arXiv:2508.00500*, 2025.

[18] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Yuanzhuo Wang, Wen Gao, Lionel Ni, and Jian Guo. A survey on LLM-as-a-judge. *arXiv preprint arXiv:2411.15594*, 2024.

[19] Lin Shi, Chiyu Ma, Wenhua Liang, Xingjian Diao, Weicheng Ma, and Soroush Vosoughi. Judging the judges: A systematic study of position bias in LLM-as-a-judge. In *Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics (AACL-IJCNLP)*, 2025. arXiv:2406.07791, 2024.

[20] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

[21] Xixun Lin, Yucheng Ning, Jingwen Zhang, Yan Dong, Yilong Liu, Yongxuan Wu, Xiaohua Qi, Nan Sun, Yanmin Shang, Kun Wang, Pengfei Cao, Qingyue Wang, Lixin Zou, Xu Chen, Chuan Zhou, Jia Wu, Peng Zhang, Qingsong Wen, Shirui Pan, Bin Wang, Yanan Cao, Kai Chen, Songlin Hu, and Li Guo. LLM-based agents suffer from hallucinations: A survey of taxonomy, methods, and directions. *arXiv preprint arXiv:2509.18970*, 2025.

[22] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[23] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630:625–630, 2024.

[24] Jannik Kossen, Jiatong Han, Muhammed Razzak, Lisa Schut, Shreshth Malik, and Yarin Gal. Semantic entropy probes: Robust and cheap hallucination detection in LLMs. *arXiv preprint arXiv:2406.15927*, 2024.

[25] Nicholas Goldowsky-Dill, Bilal Chughtai, Stefan Heimersheim, and Marius Hobbhahn. Detecting strategic deception with linear probes. In *International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, pages 19755–19786, 2025.